

Relazione Progetto: Sistema Gestione Biblioteca

Ingegneria del Software

Studente: Marco

15 Maggio 2026

Sommario

La presente relazione documenta lo sviluppo di un sistema software per la gestione di una biblioteca, focalizzandosi sull'applicazione pratica dei principi di ingegneria del software: analisi dei requisiti, modellazione UML, implementazione in Java 21 e verifica tramite testing automatizzato.

Indice

1	Introduzione	2
2	Analisi dei Requisiti	2
2.1	Requisiti Funzionali (Casi d'Uso)	2
2.2	Requisiti Non Funzionali e Criteri di Accettazione	2
3	Design del Sistema	2
3.1	Architettura e Principi SOLID	2
3.2	Matrice di Tracciabilità	3
4	Implementazione Tecnologica	3
4.1	Ambiente di Sviluppo	3
4.2	Snippet di Codice Significativo	4
5	Verifica e Validazione	4
5.1	Risultati dei Test	4
6	Conclusioni	4

1 Introduzione

Il sistema ha l'obiettivo di informatizzare i processi principali di una biblioteca universitaria, nello specifico la consultazione del catalogo e la gestione del prestito dei volumi.

2 Analisi dei Requisiti

2.1 Requisiti Funzionali (Casi d'Uso)

Il sistema risponde ai seguenti casi d'uso principali:

- **UC1 - Ricerca Libro:** Permette di interrogare il catalogo per verificare la presenza di un titolo.
- **UC2 - Richiesta Prestito:** Gestisce l'assegnazione temporanea di un libro a un utente.

2.2 Requisiti Non Funzionali e Criteri di Accettazione

Come richiesto dai vincoli comuni del progetto:

- **Affidabilità:** Il sistema garantisce l'integrità impedendo prestiti duplicati dello stesso volume.
- **Manutenibilità:** Struttura modulare con responsabilità chiare per facilitare evoluzioni future.
- **Criterio di Accettazione (Prestito):** Il prestito deve fallire (restituendo *false*) se il libro è già impegnato.
- **Criterio di Accettazione (Ricerca):** La ricerca deve avere successo anche con titoli parziali (case-insensitive).

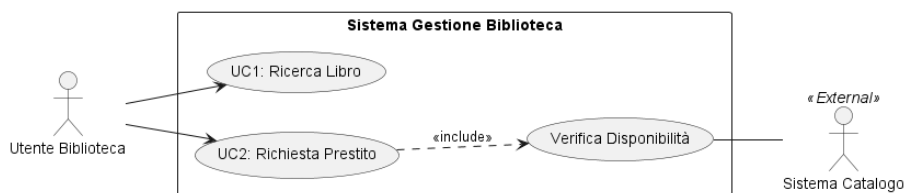


Figura 1: Diagramma dei Casi d'Uso del Sistema.

3 Design del Sistema

3.1 Architettura e Principi SOLID

L'architettura applica in modo ragionato i principi SOLID:

- **Single Responsibility Principle (SRP):** La classe `GestorePrestiti` si occupa solo della logica di transazione, separata dal modello dei dati (`Libro/Utente`).

- **Open-Closed Principle (OCP):** Il design permette di aggiungere nuove funzionalità al catalogo senza modificare il core del gestore prestiti.
- **Dependency Inversion Principle (DIP):** Le dipendenze tra moduli sono ridotte per favorire il testing unitario.

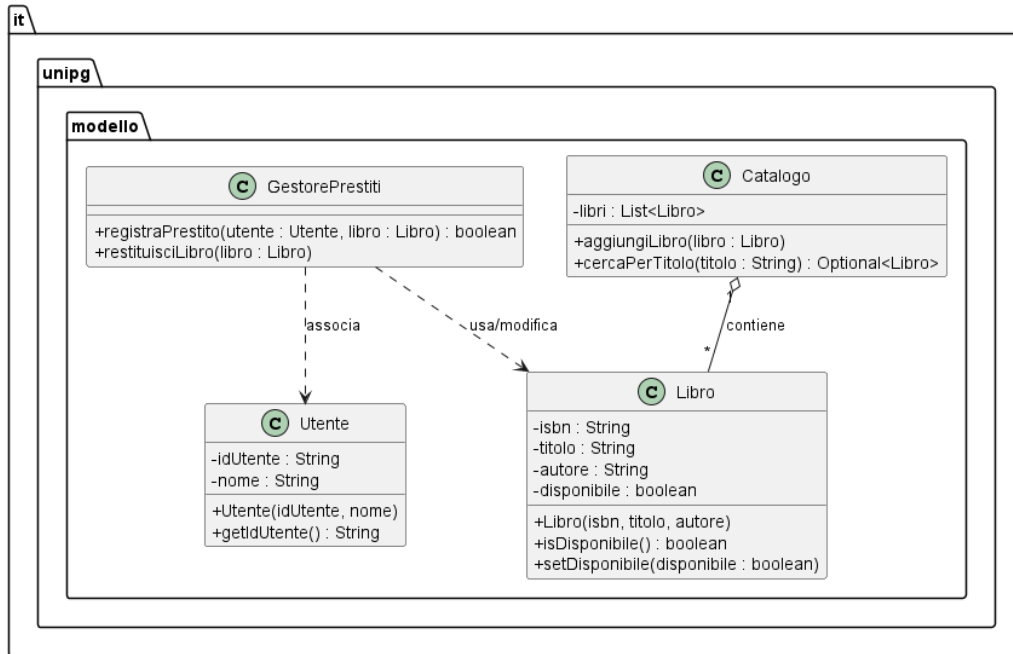


Figura 2: Diagramma delle Classi - Struttura e Relazioni.

3.2 Matrice di Tracciabilità

Per garantire il collegamento tra requisiti, codice e test:

Caso d'Uso	Implementazione	Test Unitario
UC1: Ricerca Libro	Catalogo.java	CatalogoTest.java
UC2: Richiesta Prestito	GestorePrestiti.java	GestorePrestitiTest.java

Tabella 1: Matrice di Tracciabilità.

4 Implementazione Tecnologica

4.1 Ambiente di Sviluppo

- **JDK:** Java 21 (LTS).
- **Build Tool:** Maven 3.x.
- **Testing:** JUnit 5 (Jupiter).

4.2 Snippet di Codice Significativo

```
1 public boolean registraPrestito(Utente utente, Libro libro) {  
2     if (libro.isDisponibile()) {  
3         libro.setDisponibile(false);  
4         return true;  
5     }  
6     return false;  
7 }
```

Listing 1: Logica di validazione del prestito in GestorePrestiti.java

5 Verifica e Validazione

5.1 Risultati dei Test

L'esecuzione del comando `mvn clean test` conferma il superamento dei test per flussi positivi e di errore:

- **Test eseguiti:** 4 su 4 superati (100%).
- **Stato finale:** BUILD SUCCESS.

6 Conclusioni

Il progetto soddisfa i requisiti minimi di implementazione end-to-end, qualità del design (SOLID) e documentazione richiesti.